VERSION CONTROL

Git & GitHub
Top 50 Interview Questions

Version Control System — Detailed Answers

Simple explanations with the technical terms interviewers expect

Covers VCS basics • Git internals • Core commands • Branching & Merging
Undoing changes • GitHub collaboration, Pull Requests & workflows

Quick Visual Primer

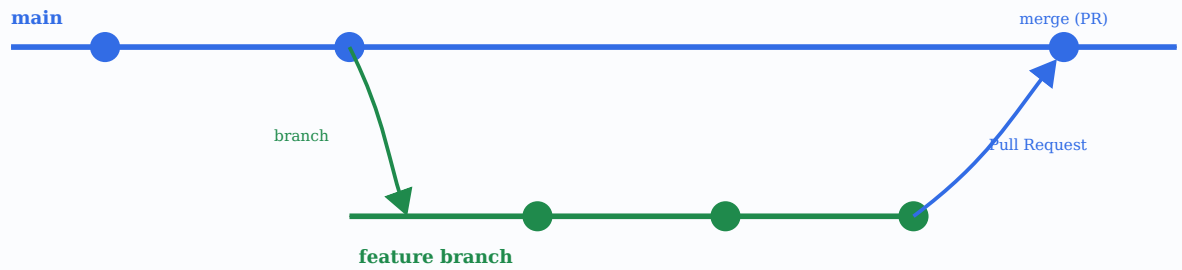
Two diagrams capture most of what Git interviews test. Keep them in mind as you read the questions.

The Three Areas of Git

Every change in Git flows through three areas. `git add` stages changes; `git commit` saves them permanently.

Branching & the Pull Request Flow

Teams isolate work on **branches** and integrate it back via **Pull Requests** after review.



Branch off main → commit on the feature branch → open a Pull Request → review → merge back into main.

Mental model: Git = the local tool that tracks history through working directory → staging → repository. GitHub = the shared hosting service where branches are merged via Pull Requests.

Top 50 Git & GitHub Interview Questions

Grouped by topic, from fundamentals to collaboration. Answers are concise but keep the technical terms; **bolded** words and `commands` are worth memorizing.

Version Control Basics

Q1. What is a Version Control System (VCS)?

Answer: A **Version Control System** is a tool that **tracks and manages changes to files over time**, allowing you to record history, revert to earlier versions, compare changes, and **collaborate** with others without overwriting each other's work. It is essential for software development.

Q2. What is the difference between centralized and distributed version control?

Answer: In a **Centralized VCS (CVCS)** like SVN, there is a **single central server** holding the repository, and clients check out files from it — if the server is down, collaboration stops. In a **Distributed VCS (DVCS)** like Git, **every developer has a full copy** of the repository (including history) locally, enabling offline work and no single point of failure.

Q3. Why is version control important in DevOps?

Answer: It provides **history, traceability, and collaboration**: teams can work in parallel, track who changed what and why, roll back faulty changes, and it is the **foundation of CI/CD** — pipelines are typically triggered by commits/merges to a repository.

Q4. What is Git?

Answer: **Git** is a free, open-source **distributed version control system** created by Linus Torvalds in 2005. It tracks changes to source code, supports **fast branching and merging**, works mostly **offline/locally**, and is the most widely used VCS today.

Q5. What is the difference between Git and GitHub?

Answer: **Git** is the **version control tool** that runs locally to track code changes. **GitHub** is a **cloud-based hosting platform** built around Git that adds collaboration features like remote repositories, pull requests, issues, code review, and CI/CD integration. In short: Git is the tool; GitHub is a service that hosts Git repositories.

Q6. Name some popular Git hosting platforms.

Answer: **GitHub**, **GitLab**, **Bitbucket**, and **Azure Repos**. They all host Git repositories and add features like pull/merge requests, issue tracking, access control, and CI/CD pipelines.

Q7. What is a repository (repo)?

Answer: A **repository** is a **storage location for a project** that contains all its files plus the complete **history of changes** tracked by Git (stored in the hidden `.git` folder). A repo can be **local** (on your machine) or **remote** (on a server like GitHub).

Git Concepts

Q8. What are the three main states/areas in Git?

Answer: Files in Git move through three areas: the **Working Directory** (where you edit files), the **Staging Area / Index** (where you place changes you intend to commit, via `git add`), and the **Repository / .git directory** (where commits are permanently stored, via `git commit`).

Q9. What is the staging area (index) and why is it useful?

Answer: The **staging area** is an **intermediate area** where you assemble the exact set of changes for your next commit. It lets you **commit selectively** — staging only some files or some changes — so commits stay logical and focused, rather than committing everything at once.

Q10. What is a commit?

Answer: A **commit** is a **snapshot of your staged changes** at a point in time, saved to the repository with a unique **SHA-1 hash ID**, an author, a timestamp, and a message. Commits form a linked history, each (except the first) pointing to its parent.

Q11. What is a SHA / commit hash?

Answer: Each commit is identified by a **40-character SHA-1 hash** (e.g. `a1b2c3...`) computed from its content and metadata. It **uniquely identifies the commit** and guarantees integrity — any change to content produces a different hash.

Q12. What is HEAD in Git?

Answer: **HEAD** is a **pointer to the current commit/branch you are working on** — usually the tip of the checked-out branch. When you commit, HEAD moves forward. A **detached HEAD** means HEAD points directly to a commit rather than a branch.

Q13. What is a branch in Git?

Answer: A **branch** is a **lightweight, movable pointer to a commit**, representing an independent line of development. Branches let you work on features or fixes **in isolation** from the main code (often `main`) and merge back when ready. Creating one is cheap and fast in Git.

Q14. What is the difference between a local and a remote repository?

Answer: A **local repository** lives on your own machine where you make commits. A **remote repository** is hosted on a server (e.g. GitHub) and is shared by the team. You sync between them with `push` (upload) and `pull` / `fetch` (download).

Q15. What is 'origin' in Git?

Answer: **origin** is the **default name Git gives to the remote repository** you cloned from. It's just a convenient alias for the remote's URL, so you can run `git push origin main` instead of typing the full URL.

Basic Commands

Q16. How do you initialize a new Git repository?

Answer: `git init` creates a new, empty Git repository in the current directory by generating a hidden `.git` folder that stores all version history.

Q17. What does 'git clone' do?

Answer: `git clone <url>` creates a **local copy of an existing remote repository**, including all files, branches, and the full commit history, and automatically sets up the remote named `origin`.

Q18. What is the difference between 'git add' and 'git commit'?

Answer: `git add` **stages** changes — moving them from the working directory to the staging area. `git commit` **records the staged changes** permanently into the repository as a new snapshot with a message. You stage first, then commit.

Q19. What does 'git status' show?

Answer: `git status` shows the **current state of the working directory and staging area** — which files are modified, staged, or untracked, and the current branch. It's the most-used command for orientation.

Q20. What is the difference between 'git fetch' and 'git pull'?

Answer: `git fetch` **downloads** new commits/refs from the remote but **does not merge** them into your working branch — it just updates your remote-tracking branches. `git pull` is **fetch + merge** — it downloads and immediately integrates the changes into your current branch.

Q21. What does 'git push' do?

Answer: `git push` **uploads your local commits to a remote repository** (e.g. `git push origin main` pushes the local `main` branch to `origin`), sharing your work with the team.

Q22. How do you view commit history?

Answer: `git log` shows the commit history (hash, author, date, message). Useful variants: `git log --oneline` (compact), `--graph` (visual branch structure), and `-n 5` (last 5 commits).

Q23. What does 'git diff' do?

Answer: `git diff` shows the **differences between changes** — by default between the working directory and the staging area. `git diff --staged` compares staged changes to the last commit, and you can diff between commits or branches too.

Q24. How do you create and switch to a new branch?

Answer: Modern way: `git switch -c feature` (or `git checkout -b feature`) creates and switches in one step. To just switch: `git switch main` (or `git checkout main`).

Q25. What does '.gitignore' do?

Answer: A **.gitignore** file lists **files and patterns Git should not track** (e.g. `node_modules/`, `*.log`, build artifacts, secrets). It keeps unnecessary or sensitive files out of the repository.

Branching & Merging

Q26. What is merging in Git?

Answer: **Merging** combines the changes from one branch into another (e.g. merging `feature` into `main` with `git merge feature`). Git creates a **merge commit** (unless it's a fast-forward) that ties the two histories together.

Q27. What is a fast-forward merge?

Answer: A **fast-forward merge** happens when the target branch has **no new commits** since the feature branch diverged — Git simply **moves the branch pointer forward** to the latest commit, with no separate merge commit needed.

Q28. What is a merge conflict and how do you resolve it?

Answer: A **merge conflict** occurs when two branches change the **same lines of a file** (or one edits a file the other deletes), so Git can't auto-merge. You resolve it by **manually editing the conflicted file** (Git marks conflicts with `<<<<<<<`, `=======`, `>>>>>>>`), then `git add` the file and commit.

Q29. What is the difference between merge and rebase?

Answer: **Merge** combines branches and **preserves history**, creating a merge commit (non-linear history). **Rebase replays your commits on top of another branch**, producing a **linear, cleaner history** but rewriting commit IDs. Rule of thumb: don't rebase commits that have already been pushed/shared.

Q30. What is 'git cherry-pick'?

Answer: `git cherry-pick <commit>` applies a **specific commit from one branch onto another**, without merging the entire branch. Useful for pulling a single bug fix into a release branch.

Q31. What is a common Git branching strategy?

Answer: Popular strategies include **Git Flow** (main, develop, feature, release, hotfix branches), **GitHub Flow** (simple: branch off `main`, open a PR, merge), and **trunk-based development** (short-lived branches merged frequently into the trunk). They standardize how teams branch and release.

Q32. What is the difference between 'git switch' and 'git checkout'?

Answer: `git checkout` is an older, multi-purpose command (switch branches AND restore files). Git later split its roles for clarity: `git switch` **for changing branches** and `git restore` **for restoring files**. They're safer because each does one job.

Q33. How do you delete a branch?

Answer: Locally: `git branch -d feature` (safe delete, only if merged) or `-D` to force. Remotely: `git push origin --delete feature`.

Q34. What is 'git stash'?

Answer: `git stash` **temporarily shelves uncommitted changes** (saving them aside) so you can switch branches or pull cleanly, then `git stash pop` reapplies them. Useful when you're mid-work and need a clean working directory.

Undoing Changes

Q35. What is the difference between 'git reset' and 'git revert'?

Answer: `git revert <commit>` creates a **new commit that undoes** a previous one — **safe for shared history**. `git reset` **moves the branch pointer back**, effectively removing commits from history — powerful but **dangerous on pushed commits** because it rewrites history.

Q36. What are the differences between 'git reset --soft', '--mixed', and '--hard'?

Answer: **--soft** moves HEAD back but **keeps changes staged**. **--mixed** (default) moves HEAD back and **unstages changes** but keeps them in the working directory. **--hard** moves HEAD back and **discards all changes** entirely (irreversible) — use with caution.

Q37. How do you undo changes in the working directory before staging?

Answer: Use `git restore <file>` (modern) or `git checkout -- <file>` (older) to discard uncommitted modifications and restore the file to its last committed state.

Q38. How do you unstage a file you added by mistake?

Answer: Use `git restore --staged <file>` (modern) or `git reset HEAD <file>` (older) to move it out of the staging area while keeping the changes in your working directory.

Q39. How do you amend the last commit?

Answer: `git commit --amend` lets you **modify the most recent commit** — change its message or add forgotten staged files. Note it **rewrites that commit**, so avoid amending commits already pushed/shared.

Q40. What is 'git reflog' and when is it useful?

Answer: `git reflog` records **where HEAD has pointed over time**, even across resets and rebases. It's a **safety net** to recover commits that seem 'lost' after a bad reset/rebase, by finding their hash and restoring them.

Q41. How do you recover a deleted branch?

Answer: Use `git reflog` to find the last commit hash of the deleted branch, then recreate it with `git branch <name> <hash>` (or `git checkout -b`).

Q42. What does 'git clean' do?

Answer: `git clean` **removes untracked files** from the working directory. Always preview with `git clean -n` (dry run) before running `git clean -f`, since it permanently deletes untracked files.

GitHub & Collaboration

Q43. What is a Pull Request (PR)?

Answer: A **Pull Request** (called a Merge Request on GitLab) is a **request to merge changes** from one branch (often a fork or feature branch) into another. It enables **code review, discussion, automated checks (CI), and approval** before the code is merged.

Q44. What is the difference between forking and cloning?

Answer: **Forking** creates a **personal server-side copy of someone else's repository** under your own account (common in open source). **Cloning** creates a **local copy on your machine** of a repository. You often **fork then clone** your fork to contribute.

Q45. What is the typical workflow to contribute to an open-source project?

Answer: **Fork** the repo → **clone** your fork locally → create a **feature branch** → make changes and commit → **push** to your fork → open a **Pull Request** to the original repo → address review feedback → maintainer **merges** it.

Q46. What is an upstream remote?

Answer: **upstream** is the conventional name for the **original repository you forked from**. You add it as a remote so you can **fetch the latest changes** from the source project and keep your fork in sync (`git fetch upstream`).

Q47. What is the difference between SSH and HTTPS for connecting to GitHub?

Answer: Both let you push/pull. **HTTPS** uses a URL with a username and token/password and works easily through firewalls. **SSH** uses a **key pair** (no password each time once set up) and is convenient for frequent use. Functionally equivalent; the choice is about authentication style.

Q48. What is a GitHub Action / what is CI/CD integration?

Answer: **GitHub Actions** is GitHub's built-in **CI/CD automation** tool. You define **workflows in YAML** that trigger on events (push, PR) to automatically **build, test, and deploy** code. It connects version control directly to the delivery pipeline.

Q49. What is a Git tag and how does it differ from a branch?

Answer: A **tag** is a **fixed pointer to a specific commit**, typically used to mark **releases** (e.g. `v1.0.0`). Unlike a **branch**, which moves as new commits are added, a tag **does not move** — it permanently marks one point in history.

Q50. Scenario: You accidentally committed a password to a public repo. What should you do?

Answer: First, **immediately rotate/revoke the exposed credential** (assume it's compromised). Then **remove it from history** using tools like `git filter-repo` or BFG Repo-Cleaner and force-push, add the file to **.gitignore**, and use **secrets management** going forward. Deleting the latest commit alone is not enough because it remains in history and others may have pulled it.
